

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет прикладної математики та інформатики

(повне найменування назва факультету)

Кафедра кібербезпеки

(повна назва кафедри)

КУРСОВА РОБОТА

на тему:

Використання фреймворку Renode для дослідження вразливостей вбудованих систем

Студентки 3 курсу, групи ПМК-31

напряму підготовки Мельничук Л. Н.

(прізвище та ініціали)

Керівник Ст. викл. Щербина М. Ю.

(посада, вчене звання, науковий ступінь, прізвище та ініціали)

Національна шкала _____

Кількість балів: _____ Оцінка: ECTS

Львів – 2026

ЗМІСТ

ЗМІСТ	2
Перелік умовних позначень, символів, скорочень і термінів	3
Вступ	4
Розділ 1. Теоретичні основи безпеки вбудованих систем	5
1.1. Архітектура вбудованих систем	5
1.2. Методи аналізу безпеки: статичне та динамічне тестування	6
1.3. Початковий завантажувач як об'єкт атаки	6
1.4. UART як «вікно» до вбудованої системи	9
1.5. Інструментарій фреймворк Renode	9
1.5.1. Огляд можливостей Renode для моделювання	10
1.5.2. Порівняння Renode з іншими емуляторами	10
1.5.3. Архітектура Renode	11
1.5.4. Інтеграція Renode зі зовнішніми інструментами	11
Розділ 2. Практична реалізація	13
2.1. Вибір цільової архітектури та налаштування середовища моделювання .	13
2.1.1. STM32F1 як представник ARM Cortex-M	13
2.1.2. MIV-RV32 як представник RISC-V	13
2.2. Скрипти емуляції для обраних архітектур	14
2.2.1. Renode-скрипт для емуляції STM32F1	14
2.2.2. Renode-скрипт для емуляції MIV_RV32	15
2.3. Перевірка середовища	15
2.4. Підготовка до атаки	16
2.5. Виконання атаки	17
Висновки	23
Список використаних джерел	24
Додатки	26

Перелік умовних позначень, символів, скорочень і термінів

ОЗП — Оперативний Запам'ятовувальний Пристрій

ОС — Операційна Система

ПЗ — Програмне забезпечення

CI — Continuous Integration

CLI — Command Line Interface

CPU — Central Processing Unit

CRC — Cyclic Redundancy Check

DMA — Direct Memory Access

ECDSA — Elliptic Curve Digital Signature Algorithm

ELF — Executable and Linkable Format

GPIO — General-purpose input/output

HAL — Hardware Abstraction Layer

I²C — Inter-Integrated Circuit, послідовний синхронний інтерфейс типу «шина»

IoT — Internet of Things, «Інтернет речей»

ISA — Instruction Set Architecture

MCU — MicroController Unit

PC — Program Counter

RAM — Random Access Memory

RTOS — Real-Time Operating System (Операційна система реального часу).

ROM — Read-Only Memory

SoC — System-on-Chip

SPI — Serial Peripheral Interface

VTOR — Vector Table Offset Register

UART — Universal Asynchronous Receiver-Transmitter

USB — Universal Serial Bus

Вступ

Renode – це фреймворк призначений для розробки, тестування, симуляції вбудованих систем. Допомагає у створенні безпечних, протестованих та ефективних пристроїв, зокрема для «Інтернету речей». Завдяки Renode розробка, тестування, налагодження та моделювання немодифікованого програмного забезпечення для пристроїв є швидким, економічно ефективним та надійним процесом. Через погану відтворюваність та неповне розуміння стану системи, тестування та розробка фізичних вбудованих систем є складною. Проте Renode розв’язує цю проблему, дозволяє запускати бінарні файли, які ідентичні до звичних прошивок плат. Інструмент моделює не лише процесори, а й System-on-Chip, дровові та бездротові з’єднання між ними.

Застосування Renode є особливо **актуальне** в часи розвитку пристроїв Інтернет речей (англ. Internet of Things, IoT). Емуляція дозволяє розв’язати проблеми з доступом до дефіцитного або дорогого пристрою. Також можна моделювати й тестувати взаємодію підсистем і роботу цілої мережі вбудованих пристроїв у реалістичних сценаріях. Сучасні процесори вбудовані у розумні системи будинків, клімат-контролю, медичні пристрої, тому дослідження таких пристроїв є критично важливим.

Метою цієї курсової роботи є теоретичне та практичне дослідження методів виявлення вразливостей у вбудованих системах за допомогою середовища Renode. Робота зосереджена на аналізі механізмів виникнення та експлуатації критичних помилок у програмному забезпеченні.

Об’єкт дослідження – процес аналізу та пошуку вразливостей вбудованих систем, методи їх тестування.

Предмет дослідження – інструментарій тестування вбудованих систем Renode, який надає змогу провести емуляцію процесорів вбудованих систем. Буде проведено пошук вразливостей та їхнє відтворення за допомогою цього інструменту.

Розділ 1. Теоретичні основи безпеки вбудованих систем

Вбудовані системи займають велику частку у сучасних технологіях. Щодня технології IoT удосконалюють різні сфери людської діяльності, тому безпека цих рішень є життєво важливою. Це стосується й промислового та медичного обладнання. У сучасних умовах московсько-української війни постала потреба в дослідженні трофейних зразків озброєння та кіберзахисті компонентів української цифрової та високоточної зброї.

1.1. Архітектура вбудованих систем

Вбудована система — це спеціалізований обчислювальний пристрій, створений для виконання конкретного завдання або обмеженого кола функцій, на відміну від універсальних комп'ютерів, призначених для широкого спектра завдань. Технічна складність та фізичний розмір варіюється від найпростіших смартгодинників до багатовузлових мереж управління виробництвом.

Оскільки такі пристрої мають обмежений програмний та апаратний функціонал, у них або відсутня операційна система, або використовується спеціалізована операційна система реального часу (RTOS), така як FreeRTOS, μ C/OS-III, VxWorks тощо. Системи можуть мати складний графічний інтерфейс взаємодії користувача або функціонувати без нього.

Деякі пристрої призначені для довгої та надійної роботи (наприклад, медичне обладнання), на відміну від ПК, котрі можуть помилятися та у більшості випадків не завдавати життєво критичної шкоди. Час виконання завдань на таких пристроях має бути чітко регламентованим, залежно від їхнього призначення. Зворотним боком високої надійності та автономності вбудованих систем часто стають обмежена пам'ять і обчислювальна потужність.

Архітектура вбудованої системи — це сукупність апаратних і програмних компонентів, які взаємодіють між собою, та зв'язків між ними [11]. Кожна система принаймні має апаратне забезпечення, в свою чергу, програмне та прикладне системне забезпечення є опційним та необов'язковим.

Найважливішою частиною вбудованих систем на апаратному рівні є процесор. Він виконує основне завдання пристрою — обчислює. Процесор має забезпечувати потрібну потужність для виконання алгоритмів. Також важливою апаратною частиною є пам'ять. Вона забезпечує зберігання як програмного забезпечення (прошивки), так і робочих даних: змінних, проміжних результатів та інформації про стан системи. Використовують енергозалежну (RAM) та енергонезалежну (ROM, EEPROM, Flash) типи пам'яті. За допомогою периферії (сенсори, різноманітні датчики, дисплеї, таймери та лічильники) відбувається комунікація із системою. Програмне забезпечення, як частина архітектури вбудованої системи, зазвичай складається з операційної системи (або середовища виконання) та прикладного коду, який забезпечує ініціалізацію, конфігурацію пристрою та обробку помилок.

Розвиток IoT зробив атаку на вбудовані системи пріоритетною ціллю для кіберзлочинців. Зловмисники шукають вразливості у ПЗ, мережевих протоколах,

апаратній частині, аналізуючи скільки часу системі потрібно для виконання певних команд, та навіть використовують методи соціальної інженерії [7].

1.2. Методи аналізу безпеки: статичне та динамічне тестування.

Статичне тестування безпеки (SAST — Static Application Security Testing) полягає у виявленні помилок та вразливостей за допомогою аналізу вбудованого вихідного коду без його виконання. Цей підхід належить до тестування “білої скриньки”. Така перевірка виконується на ранньому етапі розробки та мінімізує витрати на виправлення недоліків. Частково ефективний для виявлення переповнення буфера, дефектів криптографічного захисту та витoku чутливих даних. Цей метод не завжди є панацеєю, оскільки багато помилок і змін у поведінці виникає при виконанні програми [21]. Статичний аналіз викликає багато хибних спрацювань (false positive), що створює значний шум для тестування.

Динамічне тестування безпеки (DAST — Dynamic Application Security Testing) досліджує поведінку вбудованих систем під час виконання, виявляє вразливості пропущені статичним методом, належить до тесту “чорної скриньки”. Цей метод фактично виконує атаки, адже немає доступу до вихідного коду.

Ключовим підходом у динамічному аналізі є фаз-тестування, фазинг (fuzzing). Полягає у надсиланні до системи випадкових, неочікуваних або погано сформованих даних для виявлення змін поведінки, пошкоджень пам’яті, проблем безпеки. Фазинг спрямований на різні протоколи (мережеві, зв’язку), інтерфейси, процесори команд. Для цього методу варто використовувати середовища емуляції, такі як Renode.

Поєднання цих методів на відповідних рівнях розробки покращує ефективність тестувань кінцевих продуктів. Через ризик пошкодження фізичного обладнання, варто використовувати емуляцію віртуального середовища. Renode дозволяє проводити динамічне тестування у контрольованому середовищі та надає доступ до пам’яті та регістрів процесора.

1.3. Початковий завантажувач як об’єкт атаки

Початковий завантажувач (Bootloader) — це спеціалізований клас програмного забезпечення, що виконує критичну роль у життєвому циклі вбудованих систем і пристроїв. Його основні завдання включають початкову ініціалізацію апаратури, перевірку наявності й цілісності прошивки, передачу керування у разі успішної перевірки та оновлення прошивки за потреби, наприклад через USB або інші інтерфейси. Надійність та безпека завантажувача критично важлива, оскільки розташований на початку ланцюга завантаження та визначає, які саме програмні компоненти будуть виконуватись далі.

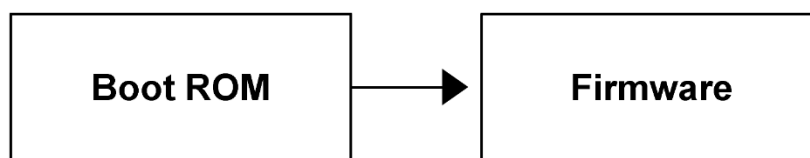


Рис. 1.3.1. Одноетапний процес завантаження прошивки

З метою виявлення випадкових помилок або навмисних змін в образі прошивки перед її виконанням використовують базовий механізм захисту – перевірка цілісності прошивки. Найпоширенішими підходами є:

- Контрольна сума — простий метод, що швидко вираховує сумарне значення байтів образу. Ефективний проти випадкових помилок, але вразливий до цілеспрямованих модифікацій.
- Циклічний надлишковий код (CRC) — наприклад, CRC32. Забезпечує краще виявлення помилок у порівнянні з простою сумою, широко використовується в прошивках і протоколах оновлення.
- Криптографічні геш-функції — наприклад, SHA-256. Дають сильні гарантії цілісності та унеможливають підробку без знання вхідних даних, особливо в поєднанні з механізмами автентифікації.

Secure Bootloader — це розширення стандартного завантажувача, яке перевіряє не лише цілісність, а й автентичність прошивки. Використовує цифрові підписи прошивки, перевірені за допомогою відкритого ключа, що вбудовані в апаратну частину або захищену область пам'яті. За допомогою ланцюга довіри (Chain of Trust) послідовно перевіряє компоненти, починаючи від первинного завантажувача до завантажувача другого рівня і до прошивки або основної операційної системи. Механізми відкату і оновлення дозволяють безпечно застосовувати нові підписи та ключі, зберігаючи гарантії цілісності.

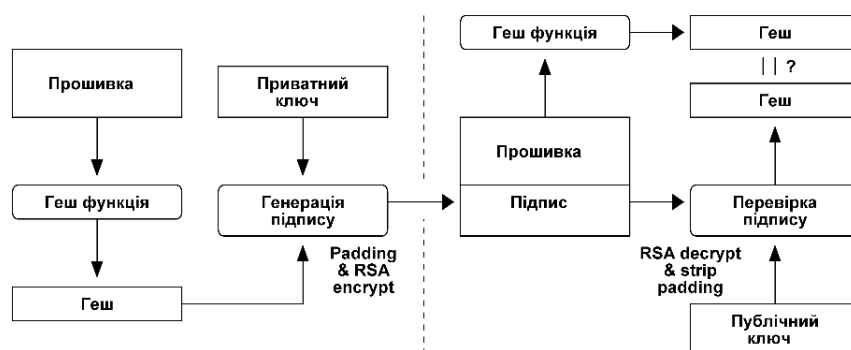


Рис. 1.3.2. Перевірка автентичності прошивки

Запровадження Secure Bootloader значно ускладнює можливість виконання неавторизованого коду на пристрої, але вимагає ретельного управління ключами та процедур оновлення.

Процес завантаження складається з послідовних етапів, кожен з яких виконує окрему перевірку або підготовчу дію.

- Ініціалізація апаратури: налаштування тактування, пам'яті, периферії та базових апаратних контролерів, необхідних для подальшого читання образу прошивки.
- Пошук і завантаження образу прошивки: визначення джерела (вбудована флеш-пам'ять, зовнішній накопичувач, інтерфейс оновлення) і завантаження образу у робочу пам'ять.
- Перевірка цілісності: обчислення контрольної суми, CRC або криптографічного гешу та порівняння з очікуваним значенням.
- Перевірка автентичності: у Secure Bootloader — перевірка цифрового підпису прошивки.
- Передача керування: якщо всі перевірки пройдені успішно, завантажувач передає керування основній прошивці.
- Механізм відновлення: у разі невдачі — запуск режиму відновлення або очікування оновлення прошивки через сервісний інтерфейс.

Завантажувач є привабливою ціллю для атак, бо виконує центральну роль у процесі ініціалізації. Мотивами атак є: отримання образу прошивки для зворотного інжинірингу та виявлення вразливостей, перепрошивка пристрою без сервісних інструментів виробника, що дозволяє встановити модифіковані образи, неавторизоване зчитування або модифікація конфіденційних даних або ключів, завантаження власних або пропатчених прошивок шляхом обходу перевірок автентичності.

Програмні атаки:

- переповнення буфера та інші вразливості пам'яті, що дозволяють виконати довільний код у контексті завантажувача;
- маніпуляції зі структурами даних прошивки або з протоколом оновлення, наприклад підміна метаданих або підробка контрольних значень;
- компрометація інструментів оновлення або ланцюга постачання, що дозволяє підмінити підписані образи ще до їх доставляння на пристрій.

Компрометація завантажувача зазвичай означає компрометацію всього подальшого ланцюга завантаження, оскільки довіра до наступних компонентів базується на перевірках, виконаних завантажувачем.

Апаратні атаки:

- Clock Glitching — короточасні маніпуляції тактовою частотою для виклику помилок у виконанні коду;
- Voltage Glitching — раптові зміни напруги живлення для створення помилок у логіці або пропуску перевірок;
- Optical Fault Injection — спрямоване освітлення чутливих ділянок кристалу для індукції помилок;
- Electromagnetic Fault Injection — використання електромагнітних імпульсів для порушення нормальної роботи апаратури.

Ці техніки можуть дозволити обійти перевірки цілісності або автентичності, особливо якщо захист реалізований програмно без апаратних бар'єрів.

1.4. UART як «вікно» до вбудованої системи

UART (Universal Asynchronous Receiver-Transmitter) — це широко вживаний асинхронний послідовний інтерфейс у вбудованих системах, призначений для обміну даними між двома пристроями без окремого тактового сигналу. Передача здійснюється по двох фізичних лініях: TX (передача) і RX (приймом), причому TX одного пристрою підключається до RX іншого і навпаки. Через відсутність синхронізуючого тактового сигналу обидва кінці повинні заздалегідь узгодити швидкість обміну (baud rate) та формат кадру, інакше приймання буде некоректним.

Формат передаваного кадру в UART включає стартовий біт, послідовність бітів даних (від 5 до 8 біт), опціональний біт парності та один або два стоп-біти. Стартовий біт сигналізує початок кадру, стоп-біти відокремлюють кадри і дають приймачу час на синхронізацію, біт парності може використовуватися для простого виявлення помилок.

Логічні рівні UART залежать від фізичної реалізації: у сучасних мікроконтролерів це зазвичай TTL/CMOS рівні (наприклад, 0/3.3 V або 0/5 V). Без апаратного або програмного контролю потоку UART при високих швидкостях може втрачати дані.

Renode надає зручні механізми інтеграції віртуальних UART-пристроїв з хостом. Емулятор дозволяє виставити `pty`-термінал (Linux/macOS) або TCP-сокет (усі платформи), підключити їх до конкретного UART у віртуальній платформі і взаємодіяти з емульованою периферією через звичні інструменти. Це дає змогу створювати гібридні сценарії, коли частина системи — реальна, а частина — симульована, або автоматизувати тести, підключаючи скрипти чи CI-середовище до сокета Renode. Renode також дозволяє перенаправляти вивід у файл.

UART широко застосовують для виводу логів і інтерактивних консолей у розробці, і часто ці інтерфейси залишаються активними в продакшн-пристроях. Завантажувачі (наприклад, U-Boot) можуть надавати CLI з командами для читання флешу й ОЗП, що дозволяє зняти образ прошивки або отримати root-доступ при фізичному підключенні. Більш комплексні атаки комбінують програмні експлойти та апаратні методи: після отримання консолі через UART злоумисник може ініціювати дамп фрагментів прошивки або ОЗП. Дослідники й пентестери регулярно демонструють такі сценарії при виявленні відкритих UART-точок.

1.5. Інструментарій фреймворк Renode

Renode — це відкритий фреймворк для розробки, який дозволяє моделювати фізичні апаратні системи, включаючи процесор, сенсори, периферію, дротове та бездротове середовище між вузлами. Проєкт розроблений компанією Antmicro

та відкрито розвивається на GitHub [17], має стабільні релізи і збірки для Linux/macOS/Windows.

1.5.1. Огляд можливостей Renode для моделювання

Renode – це програма із інтерфейсом командного рядка, яка дозволяє розгорнути широкий вибір різноманітних архітектур (ARM Cortex-M/A, RISC-V). Renode фокусується не лише на емуляції CPU, а й повного SoC: шини, UART, SPI, I²C, GPIO, DMA, таймери, мережеві інтерфейси, сенсори. Renode може імітувати цілі пристрої, що дозволяє випробувати немодифіковану прошивку в ізольованому середовищі. Можна експериментувати із програмним забезпеченням, тестувати різну конфігурацію обладнання та створювати фізичну платформу для реальних потреб. В середовищі Renode є можливість роботи з багатовузловими системами. За допомогою дротового або бездротового інтерфейсу можна підключити декілька віртуальних плат в одному середовищі моделювання, що дозволяє протестувати свої протоколи або різні сценарії використання. Стан системи можна зупиняти та налагоджувати, зберігати його та ділитись із колегами в спільному робочому середовищі. Дозволяє легко копіювати, розповсюджувати та масштабувати проєкт за потреби.

Задля ретельного тестування IoT можна створити точки зупинки та точки спостереження, які не будуть ламати поведінку програмного забезпечення. Емуляцію можна використовувати до готовності апаратного забезпечення, щоб створити ПЗ. Надає змогу проаналізувати стан кожного вузла та його периферію. Задля тестування різних векторів атак, можна застосувати різні інструменти. Можна запустити різні операційні системи на різних вузлах для перевірки власних алгоритмів. Дає змогу протестувати різні версії свого ПЗ, щоб перевірити сумісність. Для аналізу мережевого трафіку можна вводити помилки та слідкувати за результатами. Renode можна використовувати на будь-якому етапі розробки, від створення прототипу до широкого розгортання та обслуговування продукту, оскільки є гнучким та надає доступ до більшості важливих функцій.

1.5.2. Порівняння Renode з іншими емуляторами

Використання Renode для даного дослідження є найбільш доцільним, для обґрунтування цього варто розглянути інші схожі інструменти. Для порівняння будемо розглядати два інструменти: QEMU [15] та Proteus Design [14].

Quick Emulator (QEMU) – це емулятор машин та віртуалізатор. Може бути як віртуальний монітор машини або емулятор архітектури систем команд. Інструмент підтримує широкий спектр технологій віртуалізації (Linux-базовані KVM, MacOS HVM, Windows Hyper-V) та велику кількість архітектур процесорів для емуляції (x86, ARM, PowerPC, RISC-V). QEMU підтримує збереження поточного стану машин та підтримує емуляцію мережевих карт. Може взаємодіяти із хостовим прикладним забезпеченням, включаючи жорсткі диски, аудіоінтерфейси, USB пристрої, мережеві карти та CD-ROM.

В контексті дослідження вбудованих систем, QEMU має недолік, оскільки в першу чергу спрямований для запуску повноцінних операційних систем, а не моделювання мікроконтролерів. Для симуляції специфічної периферії потребує перекомпілювання вихідного коду на мові C, що є часозатратним процесом і вагомим недоліком.

Proteus Design — пакет програм для автоматизованого проєктування електронних схем. Він надає можливість для моделювання програмованих пристроїв, включаючи мікроконтролери, мікропроцесорні системи. Додатково містить розділ спрямований для проєктування та симуляції пристроїв IoT, який розрахований на пристрої базовані на Arduino, Raspberry Pi або MicroPython.

Основним недоліком Proteus Design є обмежена демонстраційна версія, а для повноцінної роботи усіх функцій потрібно придбати дорогу ліцензію.

У підсумку Renode найбільш оптимальний та ефективний вибір для дослідження вбудованих систем. Він є безкоштовним, забезпечує гнучкість у налаштуванні апаратних конфігурацій та широкий вибір архітектур у порівнянні з іншими варіантами.

1.5.3. Архітектура Renode

Renode має два розширення файлів, які є основними для роботи. Вони логічно поділені для опису апаратної частини та сценарій керування самої емуляції, що дозволяє комбінувати блоки і швидко створювати нові платформи. Інструмент підтримує завантаження ELF/HEX, GDB-підключення, трасування, збереження/відновлення стану і інтеграцію з CI [16].

Файли із розширенням .resc (Renode scripts) – це інструкція для запуску процесу емуляції. У ці файли користувач може інкапсулювати повторювальні елементи проєкту для зручного повторного використання. Сценарій містить команди для підключення відповідного апаратного файлу, завантаження бінарного файлу прошивки та запуску процесора. Renode має вбудовані готові скрипти для популярних платформ, що допомагає почати роботу з фреймворком.

За апаратну конфігурацію відповідають файли із розширенням .rpl (REnode PPlatform) — формат опису платформи. Формат є зрозумілим для фахівця, легким для читання, розширення та модифікацій. У цих файлах описується архітектура мікроконтролера, а саме: адреси оперативної пам'яті, типи процесорних ядер, підключена периферія.

Управління всім процесом відбувається в Monitor через консоль. Через нього можна запускати, зупиняти, призупиняти емуляцію, виводити вміст регістрів процесора, змінювати значення в пам'яті для перевірки вразливостей.

1.5.4. Інтеграція Renode зі зовнішніми інструментами

Розробники Renode пропонують інтеграцію із багатьма інструментами, які доповнюють процес тестування.

GDB (GNU Debugger) сервер дозволяє підключати відлагоджувачі для аналізу виконання програми. Інструмент дозволяє побачити, що відбувається всередині іншої програми під час її виконання, або що робила програма під час

збою. Для виявлення вразливостей GDB може зупиняти програму за певних умов, перевіряти, що сталося, коли програма зупинилася, змінювати параметри програми, щоб виправити наслідки однієї помилки та вивчати іншу.

Інструмент Wireshark використовується для аналізу мережеских пакетів, його можна застосувати для перехоплення мережевого трафіку між емульованими вузлами та/або мережеским інтерфейсом хоста. Renode підтримує кілька протоколів канального рівня, Ethernet, Bluetooth Low Energy, IEEE 802.15.4, CAN та дозволяє логувати весь трафік та передавати його до Wireshark.

Протокол USB/IP дозволяє підключати USB-пристрої з хоста до віртуальної плати, що емулюється в Renode.

Renode інтегрували з фреймворками для фазингу (AFL/AFL++), що дозволяє перевіряти незмінені прошивки через емуляцію апаратних інтерфейсів (UART, мережа тощо). Це дає змогу виявляти вразливості в стеку мережі, драйверах і RTOS без фізичного доступу до пристрою.

Розділ 2. Практична реалізація

2.1. Вибір цільової архітектури та налаштування середовища моделювання

Для демонстрації обрано віртуальну систему на основі мікроконтролера сімейства STM32F1 (архітектура ARM Cortex-M3) від компанії STMicroelectronics [19], і віртуальну систему на основі процесорного ядра MIV_RV32 (архітектура RISC-V) від компанії Microchip Technology [10].

Для генерації прикладів прошивок використовується крос-компілятор GCC: arm-none-eabi-gcc (версія 15.2.1) та riscv-none-elf-gcc.exe (версія 15.2.0). Оскільки компіляція здійснюється для «чистого заліза», без використання RTOS, використано open-source бібліотеку libopenm3 та HAL драйвери з miv-rv32-bootloader. Вибір libopenm3 продиктовано відносною простотою та компактністю у порівнянні з HAL драйверами середовища розробки STM32Cube від виробника. Для простоти збирання та запуску прикладів створено скрипти make_cortex-m.bat та make_risc-v.bat відповідно, які здійснюють компіляцію переданого коду мовою C, який повинен містити функцію void test(void), та запуск отриманої прошивки на емуляторі Renode. Для спілкування з хост-системою використовується популярний у вбудованих системах інтерфейс UART (швидкість 115200 бод, 8 бітів, 1 стоп-біт), який ініціалізується залежно від платформи на початку функції main(). Також згідно вимог портування стандартної бібліотеки перевизначено функцію _write(int fd, const char *buf, size_t count), яка відправляє передані дані в UART, що дозволяє використовувати однаковий код тестів для обидвох платформ, де це можливо.

2.1.1. STM32F1 як представник ARM Cortex-M

Розвиток Cortex-M почався як відповідь ARM на потребу ринку в простих, енергоефективних 32-бітних ядрах для мікроконтролерів. Першим у серії став Cortex-M3 у 2004 році, який заклав архітектуру Armv7-M з підтримкою Thumb-2, інтегрованим контролером переривань NVIC та опціональним MPU. Надалі сімейство розширювалося вниз (Cortex-M0, M0+) для дуже дешевих пристроїв і вгору (M4, M7) для задач з DSP і плаваючою точкою. Пізніші ядра додали апаратні механізми безпеки TrustZone і векторні розширення для ML. Cortex-M3 реалізує архітектуру Armv7-M з 3-ступінчастим pipeline і набором інструкцій Thumb-2, що забезпечує компактний код і високу щільність виконання.

STM32F1 — популярна серія мікроконтролерів на базі Cortex-M3 від європейського виробника напівпровідників STMicroelectronics, вважається класичною для переходу від 8/16-бітних MCU до 32-бітних рішень.

2.1.2. MIV-RV32 як представник RISC-V

RISC-V виник як академічний проєкт у Паралельній лабораторії UC Berkeley (Par Lab) під керівництвом Крсте Асановіча та Девіда Паттерсона. Перші технічні звіти та специфікації з'явилися у 2011–2014 роках, а у 2015 році управління стандартом було передано RISC-V Foundation (нині RISC-V

International). Ключова ідея — відкрита, модульна ISA без ліцензійних платежів, що дозволяє вільно реалізовувати ядра й додавати кастомні інструкції.

RISC-V визначає базові профілі і численні розширення. Базові набори включають RV32I, RV64I, RV128I, а надбудови — M (mul/div), A (атомарні), F/D/Q (плаваюча точка), C (compressed), B (bit-manipulation), V (vector) та інші спеціалізовані «Z» розширення.

RV32I — базовий 32-бітний цілий набір інструкцій, спроектований як мінімальний, але достатній для цільової компіляції та ОС. Архітектурна модель: 32 регістри x0–x31 (x0 завжди нуль), PC, фіксований 32-бітний формат інструкцій; у базі — близько 40 унікальних інструкцій.

MIV_RV32 — комерційне soft-core рішення від американського виробника напівпровідників Microchip Technology для FPGA, реалізоване як конфігурований RV32I-сумісний процесор з опціями M (mul/div), C (compressed) і навіть F (FP) у деяких версіях.

2.2. Скрипти емуляції для обраних архітектур

2.2.1. Renode-скрипт для емуляції STM32F1

```
using sysbus
```

Встановлює префікс sysbus для подальших звернень до периферії. Це дозволяє писати usart2 замість sysbus.usart2.

```
mach create
```

Створює нову віртуальну машину (machine instance). Після цього доступні об'єкти типу sysbus, cpu.

```
machine LoadPlatformDescription @platforms/cpus/stm32f103.repl
```

Завантажує опис апаратної платформи (REPL/RESC файл) для STM32F103.

```
showAnalyzer usart2
```

Підключає вікно/аналізатор для UART з ім'ям usart2 (вивід серійного порту в консоль Renode). Ім'я має точно збігатися з тим, що визначено в stm32f103.repl.

```
sysbus LoadBinary "C:\LilyCourseWork\Work\test_cortex-m.bin" 0x08000000
```

Записує бінарний образ прошивки у пам'ять за адресою 0x08000000 (типова флеш-адреса для STM32).

```
sysbus.cpu VectorTableOffset 0x08000000
```

Встановлює регістр вказівника таблиці векторів на адресу прошивки. Адреса запуску береться з VTOR.

```
start
```

Запускає емуляцію. Вивід UART буде доступний у вікні аналізатора.

2.2.2. Renode-скрипт для емуляції MIV_RV32

```
using sysbus
```

Встановлює префікс sysbus для подальших звернень до периферії.

```
mach create
```

Створює нову віртуальну машину (machine instance).

```
machine LoadPlatformDescription @platforms/cpus/miv.repl
```

Завантажує опис апаратної платформи для Microchip Mi-V (RISC-V).

```
showAnalyzer uart
```

Підключає вікно/аналізатор для UART з ім'ям. Ім'я має точно збігатися з тим, що визначено в miv.repl.

```
sysbus LoadBinary "C:\...\test_risc-v.bin" 0x60000000
```

Записує бінарний образ прошивки у пам'ять за адресою 0x60000000.

```
sysbus.cpu PC 0x60000000
```

Встановлює програмний лічильник (PC) на адресу запуску.

```
start
```

Запускає емуляцію. Вивід UART буде доступний у вікні аналізатора.

2.3 Перевірка середовища

Для перевірки середовища створено файл **test_hello.c**, який дозволяє перевірити, що наше середовище емуляції коректно працює: компілює код, запускає у Renode і перехоплює вивід UART.

Для STM32F1 вводимо команду `make_cortex-m.bat test_hello.c`



Рис. 2.3.1. Перевірка середовища Cortex-M

Для MIV_RV32 вводимо команду `make_risc-v.bat test_hello.c`

```

Renode
Renode, version 1.16.1 (d66b0c2a-202602161036)
(monitor) i $CWD/test_risc-v.resc
Starting emulation...
(machine-0) 
machine-0:sysbus.uart
Hello, world!

```

Рис. 2.3.2. Перевірка середовища RISC-V

2.4. Підготовка до атаки

Для дослідження розглянемо простий імітатор Secure Bootloader. Прошивка в даному випадку 512 байт, заповнених словом FIRMWARE. Secure Bootloader за допомогою стандартної криптохеми перевіряє її цілісність і автентичність, спочатку обчислюється SHA-256 дайджест від образу прошивки, а потім цей дайджест перевіряється за допомогою ECDSA на кривій NIST P-256. У коді це інкапсульовано викликом `p256_verify`, який приймає вміст прошивки, її розмір, підпис у форматі `r||s` і публічний ключ у форматі uncompressed EC point. Для цього використано компактну сторонню бібліотеку `p256` [12].

Було згенеровано приватний ключ на кривій і з нього експортовано публічний ключ. У виводі `openssl ec -pubout -text -noout` OpenSSL показує публічний ключ у вигляді uncompressed point: рядок **04**:... за яким йдуть 32 байти *X* і 32 байти *Y*. Їх перенесено в масив `pub[]` у коді.

Файл `fake_fw.bin` (відповідає масиву `firmware[]`) підписано командою `openssl dgst -sha256 -sign ecdsa_priv.pem -out fake_fw.sig fake_fw.bin`. Далі команда `openssl asn1parse -in fake_fw.sig -inform DER -i -dump` дістає значення *r* і *s* у шістнадцятковому вигляді, які вставлено у масив `sig[]`.

Виклик `p256_verify` у коді:

- приймає вхідні байти прошивки і її розмір, обчислює SHA-256 дайджест (всередині бібліотеки);
- інтерпретує масив `pub` як точку на кривій (*X*, *Y*) і відновлює публічний ключ;
- інтерпретує масив `sig` як 32 байти *r* і 32 байти *s*;
- використовує публічний ключ і дайджест, щоб перевірити, чи підпис відповідає приватному ключу, яким підписували;
- повертає `P256_SUCCESS` при успіху.

При невдачі виконується нескінченний цикл (симуляція `panic`), при успіху — виводиться повідомлення і виконання продовжується.

Для STM32F1 вводимо команду `make_cortex-m.bat test_sign.c p256.c`

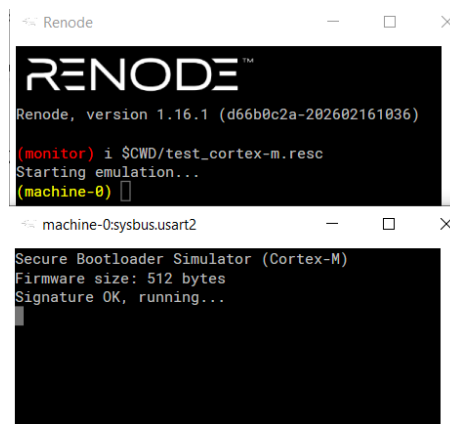


Рис. 2. 4. 1. Перевірка підпису Cortex-M

Для MIV_RV32 вводимо команду `make_risc-v.bat test_sign.c p256.c`



Рис. 2. 4. 2. Перевірка підпису RISC-V

2.5. Виконання атаки

Пропатчимо фрагмент прошивки `test_cortex-m.bin` замінивши фрагмент «FIRM» на «hack».

```
0000DB10 84 97 F1 F0 14 53 95 46 DE F4 99 EF 7C EB 45 E5  „-ср.S•FЮф™п|лЕе
0000DB20 38 D5 6B C3 46 49 52 4D 57 41 52 45 68 61 63 6B 8XkTFIRMWAREhack
0000DB30 57 41 52 45 46 49 52 4D 57 41 52 45 46 49 52 4D WAREFIRMWAREFIRM
0000DB40 57 41 52 45 46 49 52 4D 57 41 52 45 46 49 52 4D WAREFIRMWAREFIRM
```

Рис. 2.5.1. Патч фрагменту прошивки

Для перевірки, запускаємо скрипт у Renode та переконуємось, що перевірка підпису тепер неуспішна.

```

Renode
Renode, version 1.16.1 (d66b0c2a-202602161036)

(monitor) i $CWD/test_cortex-m.resc
Starting emulation...
(machine-0) 
machine-0:sysbus.usart2
Secure Bootloader Simulator (Cortex-M)
Firmware size: 512 bytes
Signature mismatch, panic!

```

Рис. 2.5.2. Перевірка неуспішного підпису Cortex-M

Проробляємо те саме для RISC-V архітектури.

```

Renode
Renode, version 1.16.1 (d66b0c2a-202602161036)

(monitor) i $CWD/test_risc-v.resc
Starting emulation...
(machine-0) 
machine-0:sysbus.usart
Secure Bootloader Simulator (RISC-V)
Firmware size: 512 bytes
Signature mismatch, panic!

```

Рис. 2.5.3. Перевірка неуспішного підпису RISC-V

Для дослідження проведемо атаку на апаратну частину (Fault Injection), щоб завантажувач сприйняв змінену прошивку як оригінальну. Розроблено скрипт `fuzzing.py` для симуляції апаратних fault injection. Він дозволяє пропустити виконання певної інструкції процесора, автоматизує цей процес і шукає вразливе місце у коді завантажувача. Скрипт послідовно ітерує адреси у заданому діапазоні. Він приймає декілька аргументів у командному рядку:

- початкова адреса (`--begin`)
- кінцевою адресою (`--end`)
- фіксованим кроком платформи (`--step`) який відповідає розміру інструкції архітектури (наприклад, 2 байти для ARM Thumb)
- шлях до конфігураційного файлу емулятора (`--resc`)
- шлях до файлу звіту (`--logfile`)
- час очікування на одну ітерацію (`--timeout`)

Ключовим механізмом симуляції збою є встановлення динамічного хука на поточну досліджувану адресу за допомогою вбудованої команди `cpu AddHook`. У момент, коли лічильник команд процесора досягає цієї точки, скрипт викликає нативний метод монітора `monitor.Parse('sysbus.cpu PC`

<new_pc>'). Ця команда миттєво перезаписує регістр PC значенням наступної адреси (**addr + step**), фактично змушуючи процесор перестрибнути поточну інструкцію без її виконання. Якщо після такого штучного збою завантажувач успішно обходить перевірку криптографічного підпису та виводить у лог-файл цільовий рядок "Signature OK, running", скрипт фіксує цю адресу як критично вразливу до атак класу Fault Injection.

Renode-скрипти **fuzz_cortex-m.resc** та **fuzz_risc-v.resc** не мають запуску, але додатково перенаправляють вивід UART у файл:

```
usart2 CreateFileBackend "C:\LilyCourseWork\Work\fuzz_cortex-m.log" true
uart CreateFileBackend "C:\LilyCourseWork\Work\fuzz_risc-v.log" true
```

відповідно. Тут true вмикає автоматичний flush, щоб дані негайно записувалися у лог без буферизації (ми аналізуватимемо цей файл).

Запускаємо команду python fuzzing.py --resc C:\LilyCourseWork\Work\fuzz_cortex-m.resc --logfile C:\LilyCourseWork\Work\fuzz_cortex-m.log --begin 0x8000154 --end 0x8000188 --step 2

```
C:\Windows\system32\cmd.exe
C:\LilyCourseWork\Work>python fuzzing.py --resc C:\LilyCourseWork\Work\fuzz_cortex-m.resc --logfile C:\LilyCourseWork\Work\fuzz_cortex-m.log --begin 0x8000172 --end 0x8000188 --step 2
[+] Testing addr 0x08000172 (skipping to 0x08000174)... -> No success
[+] Testing addr 0x08000174 (skipping to 0x08000176)... -> No success
[+] Testing addr 0x08000176 (skipping to 0x08000178)... -> SUCCESS: Signature OK
[+] Testing addr 0x08000178 (skipping to 0x0800017a)... -> No success
[+] Testing addr 0x0800017a (skipping to 0x0800017c)... -> No success
[+] Testing addr 0x0800017c (skipping to 0x0800017e)... -> No success
[+] Testing addr 0x0800017e (skipping to 0x08000180)... -> No success
[+] Testing addr 0x08000180 (skipping to 0x08000182)... -> No success
[+] Testing addr 0x08000182 (skipping to 0x08000184)... -> No success
[+] Testing addr 0x08000184 (skipping to 0x08000186)... -> No success
[+] Testing addr 0x08000186 (skipping to 0x08000188)... -> No success
[+] Testing addr 0x08000188 (skipping to 0x0800018a)... -> No success

Summary: addresses with 'Signature OK, running'
0x08000176
```

Рис. 2.5.4. Ітерація адрес STM32F1

Проаналізуємо згенерований компілятором код:

```
08000154 <test>:
8000154:    b508      push    {r3, lr}
8000156:    f44f 7200  mov.w   r2, #512    @ 0x200
800015a:    490c      ldr     r1, [pc, #48]    @ (800018c <test+0x38>)
800015c:    480c      ldr     r0, [pc, #48]    @ (8000190 <test+0x3c>)
800015e:    f003 fd15  bl      8003b8c <printf>
8000162:    4b0c      ldr     r3, [pc, #48]    @ (8000194 <test+0x40>)
8000164:    f44f 7100  mov.w   r1, #512    @ 0x200
8000168:    f103 0244  add.w   r2, r3, #68 @ 0x44
800016c:    f103 0084  add.w   r0, r3, #132   @ 0x84
8000170:    f002 f94e  bl      8002410 <p256_verify>
8000174:    2801      cmp     r0, #1
8000176:    d104      bne.n   8000182 <test+0x2e>
8000178:    e8bd 4008  ldmbia.w sp!, {r3, lr}
800017c:    4806      ldr     r0, [pc, #24]    @ (8000198 <test+0x44>)
800017e:    f003 bd79  b.w     8003c74 <puts>
8000182:    4806      ldr     r0, [pc, #24]    @ (800019c <test+0x48>)
8000184:    f003 fd76  bl      8003c74 <puts>
8000188:    f7ff ffe2  bl      8000150 <panic>
```

```

800018c:    0800da9c    .word 0x0800da9c
8000190:    0800d6e4    .word 0x0800d6e4
8000194:    0800daa8    .word 0x0800daa8
8000198:    0800d720    .word 0x0800d720
800019c:    0800d73c    .word 0x0800d73c

```

За адресою 0x08000176 стоїть інструкція умовного переходу **bne.n** (Branch Not Equal, .n - Narrow). Інструкція 16-біт з набору системних команд ARM Thumb-2, займає 2 байти. В результаті Fault Injection не виконав цієї інструкції, виконання продовжилось з нібито успішною перевіркою підпису.

Проведемо такі дії з MIV_RV32.

```

C:\Windows\system32\cmd.exe
60000a00 --step 4
[+] Testing addr 0x600009a4 (skipping to 0x600009a8)... -> No success
[+] Testing addr 0x600009a8 (skipping to 0x600009ac)... -> No success
[+] Testing addr 0x600009ac (skipping to 0x600009b0)... -> No success
[+] Testing addr 0x600009b0 (skipping to 0x600009b4)... -> No success
[+] Testing addr 0x600009b4 (skipping to 0x600009b8)... -> No success
[+] Testing addr 0x600009b8 (skipping to 0x600009bc)... -> No success
[+] Testing addr 0x600009bc (skipping to 0x600009c0)... -> No success
[+] Testing addr 0x600009c0 (skipping to 0x600009c4)... -> No success
[+] Testing addr 0x600009c4 (skipping to 0x600009c8)... -> No success
[+] Testing addr 0x600009c8 (skipping to 0x600009cc)... -> No success
[+] Testing addr 0x600009cc (skipping to 0x600009d0)... -> No success
[+] Testing addr 0x600009d0 (skipping to 0x600009d4)... -> No success
[+] Testing addr 0x600009d4 (skipping to 0x600009d8)... -> No success
[+] Testing addr 0x600009d8 (skipping to 0x600009dc)... -> No success
[+] Testing addr 0x600009dc (skipping to 0x600009e0)... -> SUCCESS: Signature OK
[+] Testing addr 0x600009e0 (skipping to 0x600009e4)... -> No success
[+] Testing addr 0x600009e4 (skipping to 0x600009e8)... -> No success
[+] Testing addr 0x600009e8 (skipping to 0x600009ec)... -> No success
[+] Testing addr 0x600009ec (skipping to 0x600009f0)... -> No success
[+] Testing addr 0x600009f0 (skipping to 0x600009f4)... -> No success
[+] Testing addr 0x600009f4 (skipping to 0x600009f8)... -> No success
[+] Testing addr 0x600009f8 (skipping to 0x600009fc)... -> No success
[+] Testing addr 0x600009fc (skipping to 0x60000a00)... -> No success
[+] Testing addr 0x60000a00 (skipping to 0x60000a04)... -> No success

Summary: addresses with 'Signature OK, running'
0x600009dc

```

Рис. 2.5.5. Ітерація адрес MIV_RV32

```

600009a4 <test>:
600009a4:    60018537    lui    a0,0x60018
600009a8:    ff010113    addi   sp,sp,-16
600009ac:    86818593    addi   a1,gp,-1944 # 80000068 <plat.0>
600009b0:    20000613    li     a2,512
600009b4:    6f050513    addi   a0,a0,1776 # 600186f0
<__clzsi2+0x8c>
600009b8:    00112623    sw     ra,12(sp)
600009bc:    219050ef    jal    600063d4 <printf>
600009c0:    600196b7    lui    a3,0x60019
600009c4:    ac468693    addi   a3,a3,-1340 # 60018ac4 <pub>
600009c8:    04468613    addi   a2,a3,68
600009cc:    08468513    addi   a0,a3,132
600009d0:    20000593    li     a1,512
600009d4:    3f4040ef    jal    60004dc8 <p256_verify>
600009d8:    00100793    li     a5,1
600009dc:    00f51c63    bne    a0,a5,600009f4 <test+0x50>
600009e0:    00c12083    lw     ra,12(sp)
600009e4:    60018537    lui    a0,0x60018
600009e8:    72c50513    addi   a0,a0,1836 # 6001872c
<__clzsi2+0xc8>
600009ec:    01010113    addi   sp,sp,16
600009f0:    3990506f    j      60006588 <puts>
600009f4:    60018537    lui    a0,0x60018
600009f8:    74850513    addi   a0,a0,1864 # 60018748
<__clzsi2+0xe4>

```

```

600009fc:    38d050ef        jal    60006588 <puts>
60000a00:    fa1ff0ef        jal    600009a0 <panic>

```

За адресою 0x600009dc знаходиться та сама інструкція переходу **bne**, яка займає 4 байти у RISC-V.

Оскільки ми тепер маємо можливість патчити прошивку, проведемо ще один експеримент. Уявімо, що

```

(void)printf("Secure Bootloader Simulator (%s)\n"
            "Firmware size: %u bytes\n", plat, fw_size);

```

відноситься до прошивки, а не до завантажувача. Знайдемо рядок формату у **test_cortex-m.bin** та змінимо його наступним чином:

```

0000D6C0  40 42 70 47 4F F0 00 00 70 47 50 EA 01 30 05 D1  @BpGOp...pGPr.O.C
0000D6D0  11 F0 00 40 08 BF 6F F0 00 40 70 47 4F F0 00 00  .p.@.iop.@pGOp..
0000D6E0  70 47 00 BF 25 70 25 70 25 70 25 70 25 70 25 70  pG.ltrtrtrtrtrtrtr
0000D6F0  25 70 25 70 25 70 25 70 25 70 25 70 25 70 25 70  trtrtrtrtrtrtrtrtrtr
0000D700  25 70 25 70 25 70 25 70 25 70 25 70 25 70 25 70  trtrtrtrtrtrtrtrtrtr
0000D710  25 70 25 70 25 70 25 70 25 70 25 70 0A 00 00 00  trtrtrtrtrtrtr....
0000D720  53 69 67 6E 61 74 75 72 65 20 4F 4B 2C 20 72 75  Signature OK, ru
0000D730  6E 6E 69 6E 67 2E 2E 2E 00 00 00 00 53 69 67 6E  nning.....Sign
0000D740  61 74 75 72 65 20 6D 69 73 6D 61 74 63 68 2C 20  ature mismatch,
0000D750  70 61 6E 69 63 21 00 00 49 4E 46 00 69 6E 66 00  panic!..INF.inf.

```

Рис. 2.5.6. Патч прошивки

Запускаємо `python fuzzing.py --resc C:\LilyCourseWork\Work\fuzz_cortex-m.resc --logfile C:\LilyCourseWork\Work\fuzz_cortex-m.log --begin 0x8000176 --end 0x8000176 --step 2 --timeout 60`

```

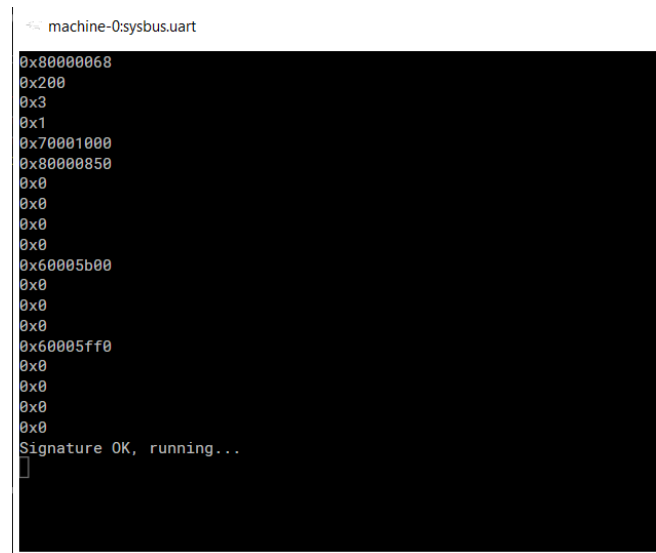
machine-0sysbus.usart2
0x800da9c
0x200
0x200c
0x200c
0x8002d7d
0x10200
0x8003abd
0x0
0x0
0x0
0x0
0x0
0x0
0x0
0x0
0x0
0x0
0x0
0x0
Signature OK, running...

```

Рис. 2.5.7. Дамп регістрів та стеку Cortex-M

Відбулося наступне: перші два числа відповідають переданим у `printf()` параметрам (регістри `r0` та `r1`), а решта — це дамп регістрів `r2`, `r3` та стеку. Зокрема, `0x8002d7d` — це адреса повернення з функції `test` у `main`.

Цей метод працює і на платформі RISC-V:



```
machine-0:sysbus.uart
0x80000068
0x200
0x3
0x1
0x70001000
0x80000850
0x0
0x0
0x0
0x0
0x60005b00
0x0
0x0
0x0
0x60005ff0
0x0
0x0
0x0
Signature OK, running...
]
```

Рис. 2.5.8. Дамп регістрів та стеку RISC-V

Обидві архітектури мають ідентичні вразливості на рівні логіки коду завантажувача. Модифікація прошивки може призвести до критичних вразливостей, за допомогою яких зловмисник може здійснити вразливий код, отримати цінну інформацію для подальшого дослідження прошивки.

Висновки

У ході виконання роботи було досліджено практично та теоретично методи виявлення вразливостей у вбудованих системах за допомогою фреймворку Renode. Проведено порівняння з подібними інструментами та обґрунтовано доцільність використання Renode.

Розглянуто динамічний та статичний методи аналізу безпеки. Також пояснено принцип роботи початкового завантажувача.

У практичній частині було проведено дослідження мікроконтролерів серії STM32F1 на базі ядра ARM Cortex-M3 та платформи MIV_RV32 на базі архітектури RISC-V. Імітовано роботу Secure Bootloader та створення цифрового підпису за допомогою бібліотеки p256 для демонстрації роботи.

З використанням скрипта фазингу проведено атаку Fault Injection на попередньо модифіковану прошивку. В результаті було виявлено вразливі адреси з інструкціями умовного переходу. Через пропуск виконання інструкції процесора вдалось обійти перевірку криптографічного підпису.

За допомогою експлуатації вразливості рядка формату вдалось продемонструвати витік даних із регістрів процесора та стеку, зафіксувавши, зокрема, адресу повернення.

Результати практичної частини підтвердили ефективність застосування середовища емуляції Renode. Без використання фізичних пристроїв вдалось дослідити вразливості коду для обидвох моделей мікроконтролерів.

Список використаних джерел

1. Щербина М. Ю. Вразливості початкового завантажувача у мікроконтролерах з SPI флеш-пам'яттю / М. Ю. Щербина, П. С. Венгерський // ITSec: Безпека інформаційних технологій : Матер. XIV Міжнар. наук.-тех. конф., Тернопіль, Україна, 22-24 трав. 2025 р. – Тернопіль-Київ : ЗУНУ-ДУІКТ, 2025. – С. 220-222. – URL: <https://drive.google.com/file/d/1Nt2w9E-N97TAIdAGqWKbaXsqASyP-4tt/view?usp=sharing> (дата звернення: 23.05.2026).
2. **Creation and Research of Concepts of Remote Firmware Update in STM32 Microcontrollers** / А. Борщов, О. Зубков // Харківський національний університет радіоелектроніки. – 2025. – № 013. URL: <https://mts.nure.ua/wp-content/uploads/2025/11/013.pdf> (дата звернення: 23.05.2026).
3. **Den Herrewegen J.V.** Fill your Boots: Enhanced Embedded Bootloader Exploits via Fault Injection and Binary Analysis / J.V. den Herrewegen, D. Oswald, F.D. Garcia, Q. Temeiza // IACR Transactions on Cryptographic Hardware and Embedded Systems. – 2020. – Vol. 2021, № 1. – P. 56–81.
4. **Design and Simulation of Embedded Microcontroller Systems** / University of Boumerdes Repository. 2022. URL: <https://dspace.univ-boumerdes.dz/server/api/core/bitstreams/e650a2af-b09a-4afe-ae81-9955fc200fef/content> (дата звернення: 23.05.2026).
5. **Dynamic Verification of Embedded Software Using Renode Framework** / MDPI. Engineering Proceedings. – 2024. – Vol. 79, Issue 1. – P. 52. URL: <https://www.mdpi.com/2673-4591/79/1/52> (дата звернення: 23.05.2026).
6. **Heath S.** Embedded Systems Design. 2nd ed. / Newnes, 2003. 419 p. URL: [http://diliev.com/home/applications/Library/Embedded%20Systems%20Design%202nd%20ed%20-%20S%5B1%5D.%20Heath%20\(Newnes,%202003\)%20WW.pdf](http://diliev.com/home/applications/Library/Embedded%20Systems%20Design%202nd%20ed%20-%20S%5B1%5D.%20Heath%20(Newnes,%202003)%20WW.pdf) (дата звернення: 23.05.2026).
7. **How Hackers Are Targeting Embedded Systems** / Risk and Resilience Hub. 2021. URL: <https://riskandresiliencehub.com/how-hackers-are-targeting-embedded-systems/> (дата звернення: 23.05.2026).
8. **Introduction to Renode for Embedded System Simulation and Testing** / Memfault. Interrupt Blog. 2022. URL: <https://interrupt.memfault.com/blog/intro-to-renode> (дата звернення: 23.05.2026).
9. **libopencm3: Open Source ARM Cortex-M Microcontroller Library** / GitHub Repository. URL: <https://github.com/libopencm3/libopencm3> (дата звернення: 23.05.2026).
10. **Microchip Technology Inc.** MIV-RV32 IP core tools [Електронний ресурс]. URL: <https://www.microchip.com/en-us/products/fpgas-and-plds/ip-core-tools/miv-rv32> (дата звернення: 23.05.2026).
11. **Noergaard T.** Embedded Systems Architecture: A Comprehensive Guide for Engineers and Programmers. Newnes, 2012. 658 p. URL: https://biorobotics.fi-p.unam.mx/wp-content/uploads/dlm_uploads/2020/02/Embedded-Systems-Architecture-A-Comprehensive-Guide-for-Engineers-and-Programmers.pdf (дата звернення: 23.05.2026).
12. **p256: Lightweight ECDSA for Embedded Systems** / O. Reparaz // GitHub Repository. URL: <https://github.com/oreparaz/p256> (дата звернення: 23.05.2026).

13. **Peckol J. K.** Embedded System Design: Introduction to SoC System Architecture. Wiley, 2010. 498 p. URL: <https://www.embedic.com/uploads/files/20201009/Embedded%20System%20Design%20Introduction%20to%20SoC%20System%20Architecture.pdf> (дата звернення: 23.05.2026).
14. **Proteus Design Suite: Simulation and PCB Design** / Labcenter Electronics. URL: <https://www.labcenter.com/simulation/> (дата звернення: 23.05.2026).
15. **QEMU Documentation** / QEMU Project. URL: <https://www.qemu.org/docs/> (дата звернення: 23.05.2026).
16. **Renode Documentation** / Antmicro. Read the Docs. URL: <https://renode.readthedocs.io/en/latest/index.html> (дата звернення: 23.05.2026).
17. **Renode framework repository** / Antmicro. GitHub. URL: <https://github.com/renode/renode> (дата звернення: 23.05.2026).
18. **Security Analysis and Fault Injection Simulation in Embedded Systems** / National Center for Biotechnology Information. PMC. 2023. Vol. 10674897. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10674897/> (дата звернення: 23.05.2026).
19. **STMicroelectronics**. STM32F1 series [Електронний ресурс]. URL: <https://www.st.com/en/microcontrollers-microprocessors/stm32f1-series.html> (дата звернення: 19.05.2026).
20. **Venherskyi P.** Mitigating Secure Bootloader Vulnerabilities in Flashless Microcontrollers / P. Venherskyi, M. Scherbyna // Вісник Львівського університету. Серія прикладна математика та інформатика. – 2025. – Вип. 32. – Р. 55-60.
21. **Why Static Analysis is Not Enough for Embedded Software Security** / Code Intelligence. 2024. URL: <https://www.code-intelligence.com/blog/embedded-software-why-static-analysis-is-not-enough> (дата звернення: 23.05.2026).

Додатки

test_cortex-m.resc

```
using sysbus
mach create
machine LoadPlatformDescription @platforms/cpus/stm32f103.repl
showAnalyzer usart2
sysbus LoadBinary "C:\LilyCourseWork\Work\test_cortex-m.bin" 0x08000000
sysbus.cpu VectorTableOffset 0x08000000
start
```

test_risc-v.resc

```
using sysbus
mach create
machine LoadPlatformDescription @platforms/cpus/miv.repl
showAnalyzer uart
sysbus LoadBinary "C:\LilyCourseWork\Work\test_risc-v.bin" 0x60000000
sysbus.cpu PC 0x60000000
start
```

test_hello.c

```
/* Мінімальний тестовий модуль */

#include <stdio.h>

void test(void) {
    /* Виводимо просте повідомлення – перевірка, що код виконується */
    (void)puts("Hello, world!");
}
```

make_cortex-m.bat

```
@echo off
set PATH=C:\LilyCourseWork\Tools\xpack-arm-none-eabi-gcc-15.2.1-1.1\bin;C:\LilyCourseWork\Tools\xpack-arm-none-eabi-gcc-15.2.1-1.1\libexec\gcc\arm-none-eabi\15.2.1;C:\LilyCourseWork\Tools\msys64\mingw64\bin;C:\LilyCourseWork\Tools\Renode
arm-none-eabi-gcc.exe -O2 -Wall -mcpu=cortex-m3 -nostartfiles -Ilibopencm3\include -Wl,-l:libopencm3\lib -Wl,-Ttest_cortex-m.ld -DSTM32F1 %* test_cortex-m.c -lopencm3_stm32f1 -o test_cortex-m.elf
if errorlevel 1 goto quit
arm-none-eabi-objdump.exe -S test_cortex-m.elf >test_cortex-m.txt
arm-none-eabi-objcopy.exe test_cortex-m.elf -Obinary test_cortex-m.bin
renode test_cortex-m.resc
:quit
```

make_risc-v.bat

```
@echo off
set PATH=C:\LilyCourseWork\Tools\xpack-riscv-none-elf-gcc-15.2.0-1\bin;C:\LilyCourseWork\Tools\xpack-riscv-none-elf-gcc-15.2.0-1\riscv-none-elf\bin;C:\LilyCourseWork\Tools\Renode
```



```

    'F','I','R','M','W','A','R','E','F','I','R','M','W','A','R','E',
    'F','I','R','M','W','A','R','E','F','I','R','M','W','A','R','E',
    'F','I','R','M','W','A','R','E','F','I','R','M','W','A','R','E',
    'F','I','R','M','W','A','R','E','F','I','R','M','W','A','R','E',
    'F','I','R','M','W','A','R','E','F','I','R','M','W','A','R','E',
    'F','I','R','M','W','A','R','E','F','I','R','M','W','A','R','E'
};

/* Публічний ключ у форматі uncompressed EC point:
 * 0x04 || X (32 байти) || Y (32 байти)
 */
static const uint8_t pub[] = {
    /* constant */
    0x04u,
    /* x-coord */
    0xF8u, 0xCBu, 0x09u, 0x27u, 0xC2u, 0xD0u, 0x2Cu, 0xBFu,
    0x98u, 0x4Bu, 0x96u, 0x71u, 0xC1u, 0xEDu, 0xF8u, 0x9Fu,
    0x22u, 0x4Cu, 0xACu, 0xF6u, 0x57u, 0x7Eu, 0xD5u, 0xEDu,
    0x09u, 0xBEu, 0xE7u, 0x2Bu, 0x80u, 0xF6u, 0x1Eu, 0x13u,
    /* y-coord */
    0x58u, 0x03u, 0x5Cu, 0xB8u, 0xE1u, 0x7Bu, 0xFEu, 0x2Bu,
    0x9Bu, 0x18u, 0x8Du, 0x9Au, 0x26u, 0x51u, 0xFB u, 0xD9u,
    0x0Eu, 0xEDu, 0xC7u, 0x49u, 0x37u, 0x3Bu, 0x6Au, 0x08u,
    0x05u, 0xF8u, 0x1Bu, 0x88u, 0x88u, 0x04u, 0x2Du, 0x48u
};

/* Підпис у форматі raw r||s (32 байти r, потім 32 байти s).
 * Бібліотека p256 очікує саме такий формат для перевірки.
 */
static const uint8_t sig[] = {
    /* r */
    0x63u, 0xFFu, 0xC1u, 0xA7u, 0x52u, 0x39u, 0xCCu, 0x6Bu,
    0xF9u, 0x5Cu, 0x2Du, 0x9Au, 0xC5u, 0x54u, 0xF0u, 0xEFu,
    0x76u, 0x20u, 0x42u, 0x40u, 0xFBu, 0xFCu, 0x71u, 0x47u,
    0xD2u, 0x7Au, 0x63u, 0x5Du, 0xCAu, 0x5Bu, 0x16u, 0x27u,
    /* s */
    0x46u, 0x7Eu, 0xDEu, 0xCAu, 0xEFu, 0xA4u, 0xFD u, 0x90u,
    0x65u, 0xFCu, 0x68u, 0xE6u, 0x84u, 0x97u, 0xF1u, 0xF0u,
    0x14u, 0x53u, 0x95u, 0x46u, 0xDEu, 0xF4u, 0x99u, 0xEFu,
    0x7Cu, 0xEBu, 0x45u, 0xE5u, 0x38u, 0xD5u, 0x6Bu, 0xC3u
};

/* Безповоротна функція аварійної зупинки.
 * noinline - забороняє інлайн для єдиної точки аварійної зупинки.
 * noreturn - підказка компілятору, що функція ніколи не повертає.
 */
static void __attribute__((noinline, noreturn)) panic(void) {
    for (;;) {}
}

/*
 * Тестова функція, що імітує поведінку bootloader:
 * - Обчислює дайджест SHA-256 від прошивки (всередині p256_verify)
 * - Перевіряє ECDSA підпис (P-256) за допомогою публічного ключа
 * - У разі невідповідності - зупиняє виконання (panic)
 * - У разі успіху - продовжує виконання (символічно виводить повідомлення)

```

```

*/
void test(void) {
    static const char plat[] =
#ifdef __riscv
        "RISC-V";
#elif defined(__thumb__)
        "Cortex-M";
#else
#error
#endif
    size_t fw_size = sizeof(firmware);
    (void)printf("Secure Bootloader Simulator (%s)\n"
               "Firmware size: %u bytes\n", plat, fw_size);
    if (p256_verify((uint8_t *)firmware, fw_size,
                   (uint8_t *)sig, pub) == P256_SUCCESS) {
        (void)puts("Signature OK, running...");
        return; /* у справжньому бути запускаємо firmware */
    }
    (void)puts("Signature mismatch, panic!");
    panic();
}

```

fuzzing.py

```

#!/usr/bin/env python3
import argparse
import subprocess
import time
import re
from pathlib import Path
RENODE_EXE = r"C:\LilyCourseWork\Tools\Renode\renode.exe"

def parse_int(s):
    s = s.strip().lower()
    if s.startswith("0x"):
        return int(s, 16)
    return int(s, 0)

def run_once(renode_exe, resc_path, logfile_path, addr_hex, new_pc_hex,
            timeout=8):
    # Видаляємо попередній лог, щоб читати тільки результати поточної ітерації
    if logfile_path.exists():
        try:
            logfile_path.unlink()
        except OSError:
            pass

    # Виконуємо нативну команду CLI Renode: sysbus.cpu PC 0x...
    hook_cmd = f"cpu AddHook {addr_hex} \"monitor.Parse('sysbus.cpu PC {new_pc_hex}')\""

    # Запуск Renode у фоновому процесі
    proc = subprocess.Popen([renode_exe,
                              "-e", f"include \"{str(resc_path)}\"",
                              "-e", hook_cmd,
                              "-e", "start"],
                            stdout=subprocess.PIPE, stderr=subprocess.STDOUT)

```

```

try:
    proc.wait(timeout=timeout)
except subprocess.TimeoutExpired:
    proc.terminate()
    proc.wait(timeout=5)

# Даємо час операційній системі скинути буфери у файл
time.sleep(0.5)

if not logfile_path.exists():
    return False, "Log file not created"

text = logfile_path.read_text(errors="ignore")
found = re.search(r"Signature OK, running", text)
return bool(found), text

def main():
    p = argparse.ArgumentParser(description="Run Renode over address range with
dynamic PC skipping via Monitor")
    p.add_argument("--resc", "-r", required=True, help="Path to .resc to
include")
    p.add_argument("--logfile", "-l", required=True, help="Path to the single
logfile, e.g. C:/logs/renode.log")
    p.add_argument("--begin", "-b", required=True, help="Begin address (decimal
or 0xHEX)")
    p.add_argument("--end", "-e", required=True, help="End address (decimal or
0xHEX), inclusive")
    p.add_argument("--step", "-s", required=True, help="Step/instruction size
in bytes (decimal or 0xHEX)")
    p.add_argument("--timeout", type=int, default=11, help="Timeout seconds per
run")
    args = p.parse_args()

    resc = Path(args.resc)
    if not resc.exists():
        raise SystemExit(f"RESC not found: {resc}")

    logfile = Path(args.logfile)
    logfile.parent.mkdir(parents=True, exist_ok=True)

    begin = parse_int(args.begin)
    end = parse_int(args.end)
    step = parse_int(args.step)

    found = []

    for addr in range(begin, end + 1, step):
        addr_hex = f"0x{addr:08x}"

        # Обчислюємо адресу для стрибка
        new_pc = addr + step
        new_pc_hex = f"0x{new_pc:08x}"

        print(f"[+] Testing addr {addr_hex} (skipping to {new_pc_hex})...",
end="", flush=True)

```

```

    ok, text_or_msg = run_once(RENODE_EXE, resc, logfile, addr_hex,
new_pc_hex, timeout=args.timeout)

    if ok:
        print(f" -> SUCCESS: Signature OK")
        found.append(addr_hex)
    else:
        print(f" -> No success")

    print("\nSummary: addresses with 'Signature OK, running'")
    if found:
        for a in found:
            print(a)
    else:
        print("None found")

if __name__ == "__main__":
    main()

```

fuzz_cortex-m.resc

```

using sysbus
mach create
machine LoadPlatformDescription @platforms/cpus/stm32f103.repl
usart2 CreateFileBackend "C:\LilyCourseWork\Work\fuzz_cortex-m.log" true
showAnalyzer usart2
sysbus LoadBinary "C:\LilyCourseWork\Work\test_cortex-m.bin" 0x08000000
sysbus.cpu VectorTableOffset 0x08000000

```

fuzz_risc-v.resc

```

using sysbus
mach create
machine LoadPlatformDescription @platforms/cpus/miv.repl
uart CreateFileBackend "C:\LilyCourseWork\Work\fuzz_risc-v.log" true
showAnalyzer uart
sysbus LoadBinary "C:\LilyCourseWork\Work\test_risc-v.bin" 0x60000000
sysbus.cpu PC 0x60000000

```